

RELATÓRIO TÉCNICO

Sistema de Cadastro Escolar

Disciplina: Introdução às Técnicas de Programação

Unidade: 2

Aluno: Breno Igor da Silva

Matrícula: 20250065477

Período: 2025.2

1. INTRODUÇÃO E CONTEXTO

1.1 Nome do Projeto e Evolução

O Sistema de Cadastro Escolar foi expandido na U2 com novos conceitos de programação. O sistema continua gerenciando matrículas de alunos do ensino fundamental, mas agora conta com funcionalidades mais avançadas de busca, visualização de dados e gerenciamento de memória.

1.2 Objetivos da U2

- Implementar sistema de busca avançada usando strings
- Criar matriz bidimensional para visualização de dados
- Aplicar ponteiros para otimizar passagem de dados
- Utilizar loops aninhados para estruturas bidimensionais

1.3 Funcionalidades Implementadas

- Busca por nome completo usando strcmp
- Busca por série e turma específica
- Busca por turno organizada por série
- Matriz de distribuição de alunos por série e turma

2. ANÁLISE TÉCNICA

2.1 Metodologia - Ferramentas Utilizadas

Sistema Operacional: Linux

Compilador: GCC versão 13.3.0

Editor de Código: Visual Studio Code

Linguagem: C

2.2 Aplicação dos Conceitos da U2

2.2.1 Strings

Implementei a função strcmp() da biblioteca string.h para comparar nomes na busca. A função retorna 0 quando as strings são iguais:

```
if(strcmp(lista[i].nome, nome) == 0){  
    // Aluno encontrado  
}
```

Arrays de char continuam armazenando nomes (até 50 caracteres) e dados dos responsáveis, com scanf usando %49[^\n] para ler linhas completas.

2.2.2 Estruturas de Repetição Aninhadas

Usei loops aninhados em três situações principais:

Matriz de distribuição: Loop externo percorre séries, interno percorre turmas

Busca por turno: Primeiro loop nas séries, segundo nos alunos daquela série

Atualização da matriz: Percorre alunos e atualiza posições na matriz

2.2.3 Matrizes

A matriz distribuicao_turmas[MAX_SERIES][MAX_TURMAS] é um array bidimensional onde:

Linhas (0-8) = séries (1ª a 9ª)

Colunas (0-1) = turmas (A e B)

Cada célula = quantidade de alunos

Operações: inicialização, atualização automática, exibição em tabela, totalização por linha e coluna.

Percorrimento: Loops aninhados acessam os elementos. Índice da série é serie - 1, índice da turma é 0 para A e 1 para B.

2.2.4 Ponteiros

Todas as funções de busca recebem o array como ponteiro:

```
void buscar_por_nome(Aluno *lista, int total)
```

Isso passa apenas o endereço ao invés de copiar o array inteiro, economizando memória e tornando o acesso mais rápido. Dentro das funções uso lista[i] para acessar elementos.

3. IMPLEMENTAÇÃO E REFLEXÃO

3.1 Dificuldades Encontradas

1. **Entender ponteiros:** minha maior dificuldade foi entender bem ponteiros que eu ainda não havia estudado, e saber como implementar isso no projeto de forma clara e útil.

3.2 Soluções Implementadas

1. **Para ponteiros:** Pratiquei bastante e percebi que arrays já são ponteiros naturalmente. Escrever comentários explicando ajudou bastante.

3.3 Organização do Código

- Definições e structs no topo
- Variáveis globais (ponteiro para alunos)
- Funções de busca da U2
- Funções de matriz
- Funções originais da U1
- Função main com menu

3.4 Conclusão

Aprendizados obtidos:

Aplicar ponteiros na prática deixou mais claro o entendimento e o poder deles.

As matrizes me mostraram na prática como organizar dados de forma visual e intuitiva.

Os loops aninhados também se tornam mais naturais ao trabalhar com estruturas bidimensionais.